



Conectar un sitio web a
una base de datos
utilizando PDO

PDO: Objetos de Datos de PHP

- ▶ La extensión *Objetos de Datos de PHP* define una interfaz para poder acceder a bases de datos en PHP.
- ▶ PDO proporciona una capa de abstracción de *acceso a datos*, lo que significa que, independientemente del gestor de bases de datos que se esté utilizando, se emplean las mismas funciones para realizar consultas y obtener datos.
- ▶ PDO permite acceder a diferentes sistemas gestores de bases de datos **con un controlador específico** (MySQL, MariaDB, SQLite, Oracle...) mediante el cual se conecta.

Ejecuta `phpinfo()` para saber que controladores están instalados en tu servidor web.

PDO: Objetos de Datos de PHP

- ▶ Considerad PDO como un conjunto de clases que vienen empaquetadas con PHP para facilitarle la interacción con su base de datos.
- ▶ Cuando desarrollamos una aplicación PHP que trabaje con una Base de datos necesitamos resolver diferentes problemas:
 - Establecer una conexión con la BD
 - Crear una consulta
 - Utilizar el recurso de conversión de resultados devueltos por la BD en un array asociativo u otra estructura de datos.
 - Evitar la inyección SQL, utilizando **sentencias preparadas**.

Inyección SQL

- ▶ La inyección SQL, o SQLi, es un tipo de ataque a una aplicación web que permite a un atacante insertar sentencias SQL maliciosas en la aplicación web, obteniendo potencialmente acceso a datos sensibles en la base de datos o destruyendo estos datos, la inyección SQL fue descubierta por primera vez por Jeff Forristal en 1998.
- ▶ En las dos décadas que han transcurrido desde su descubrimiento, la inyección SQL ha sido sistemáticamente la principal prioridad de los desarrolladores web a la hora de crear aplicaciones.

`select * from usuarios where nombre='luis' and clave='$variable';`

- ▶ Imagina la típica aplicación web que implica solicitudes de bases de datos a través de las entradas del usuario y la entrada se realiza a través de un formulario, donde pide usuario y clave.
- ▶ El usuario introduce en el campo **clave** del formulario: **`'98756' or '1=1'`**
- ▶ La instrucción sql quedaría:

`select * from usuarios where nombre='luis' and clave='98756' or '1=1';`

PDO: Objetos de Datos de PHP

El sistema PDO se fundamenta en 3 clases: **PDO**, **PDOStatement** y **PDOException**:

- ▶ La **clase PDO** se encarga de mantener la conexión a la base de datos y otro tipo de conexiones específicas como transacciones, además de crear instancias de la clase PDOStatement.
- ▶ La **clase PDOStatement**, es la que maneja las sentencias SQL y devuelve los resultados.
- ▶ La **clase PDOException** se utiliza para manejar los errores.

Clase PDO

Clase PDO

Representa **una conexión entre PHP y un servidor de bases de datos.**

► Crea un objeto de la clase PDO para representar una conexión a la base de datos solicitada.

PDO::__construct

```
public PDO::__construct(  
    string $dsn,  
    ?string $username = null,  
    ?string $password = null,  
    ?array $options = null )
```

Clase PDO

`string $dsn`

un DSN consiste en el nombre del controlador de PDO, en nuestro caso `mysql`, seguido por dos puntos, y a continuación la sintaxis específica del controlador de PDO para la conexión:

```
$dsn = 'mysql:host=localhost;dbname=prueba1;port=3306;charset=utf8';
```


Clase PDO

Ejemplo de conexión

```
const HOST='localhost';
const DATABASE='prueba1';
const USERNAME='root';
const PASSWORD='';
const PORT='3306';//puerto en el que escucha MariaDB
const CHARSET='UTF8';
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
```

Clase PDOException

Excepciones

El método `setAttribute()` de la clase PDO se utiliza para establecer atributos de la conexión a la base de datos. Este método permite modificar el comportamiento del objeto PDO en tiempo de ejecución y se utiliza para configurar el manejo de errores, entre otras opciones

```
public PDO::setAttribute(int $attribute, mixed $value): bool
```

- **\$attribute**: Es el atributo que deseas establecer. Existen varios atributos predefinidos que puedes usar, como `PDO::ATTR_ERRMODE`, `PDO::ATTR_DEFAULT_FETCH_MODE`, entre otros.
- **\$value**: Es el valor que deseas asignar al atributo especificado.

PDO::ATTR_ERRMODE y PDO::ERRMODE_EXCEPTION

Cuando se establece el atributo PDO::ATTR_ERRMODE con el valor PDO::ERRMODE_EXCEPTION, se indica que deseas que PDO lance una excepción PDOException cada vez que ocurre un error en la operación de la base de datos.

Manejo de Errores: Al lanzar excepciones en lugar de solo devolver códigos de error, puedes manejar los errores de manera más efectiva mediante bloques try-catch. Esto te permite capturar detalles sobre el error, como el mensaje de error, el código de error, el fichero del error..., lo que es útil para depuración.

Seguridad: Al utilizar excepciones, evitas exponer información sensible sobre la estructura de la base de datos. Cuando un error ocurre, puedes personalizar cómo se informa y se registra el error, evitando mostrar información que podría ser utilizada por un atacante.

```
<?php
// Conexión a la base de datos
const HOST = 'localhost';
const DATABASE = 'probando';
const USERNAME = 'root';
const PASSWORD = '';
const PORT = '3306'; //puerto en el que escucha MariaDB
const CHARSET = 'UTF8';

try {
    $dns = "mysql:host=" . HOST . ";dbname=" . DATABASE . ";port=" . PORT . ";charset=" . CHARSET;

    // $conexion contiene el objeto PDO que representa la conexión a la base de datos
    $conexion = new PDO($dns, USERNAME, PASSWORD);

    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {

    echo "<pre>Error: " . $e->getMessage() . "<br>";
    echo "Código de error: " . $e->getCode() . "<br>";
    echo "Archivo: " . $e->getFile() . "<br>";
    echo "Línea: " . $e->getLine() . "<br>";
    echo $cadena = print_r($e->getTrace(), true); //Array errores, convertido a string para visualizarlo
    echo "Rastro: " . $e->getTraceAsString(). "</pre>";

    // Archivo: El nombre del archivo donde ocurrió cada llamada.
    // Línea: El número de línea en el archivo donde ocurrió cada llamada.
    // Función: El nombre de la función o método que fue llamado.
    // Argumentos: Los argumentos que fueron pasados a la función o método
    exit;
}
```

```

$logFile = "miFicheroErrores.log";
// Eliminar el archivo de log si existe, para que no se acumulen los errores
if (file_exists($logFile)) {
    unlink($logFile);
}
//$conexion = new PDO('mysql:host=' . HOST . ';dbname=' . DATABASE . ';port=' . PORT . ';charset=' . CHARSET, USERNAME, PASSWORD);
try {
    $dns = "mysql:host=" . HOST . ";dbname=" . DATABASE . ";port=" . PORT . ";charset=" . CHARSET;
    // $conexion contiene el objeto PDO que representa la conexión a la base de datos
    $conexion = new PDO($dns, USERNAME, PASSWORD);
    // Establecer el modo de error de PDO a excepción
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    // Consulta de prueba
    $sql = "SELECT * FROM usuarios";
    // Ejecutar la consulta. $resultado contiene el objeto PDOStatement con los resultados
    $resultado = $conexion->query($sql);
    // Obtener los resultados
    $usuarios = $resultado->fetchAll(PDO::FETCH_ASSOC);
    var_dump($usuarios);
} catch (PDOException $e) {
    echo "<pre>Error: " . $e->getMessage() . "<br>";
    echo "Código de error: " . $e->getCode() . "<br>";
    echo "Archivo: " . $e->getFile() . "<br>";
    echo "Línea: " . $e->getLine() . "<br>";
    echo "Rastro: " . $e->getTraceAsString() . "</pre>";
    // Rastro: Una lista de todas las llamadas que se hicieron para llegar a la excepción.
    // Archivo: El nombre del archivo donde ocurrió cada llamada.
    // Línea: El número de línea en el archivo donde ocurrió cada llamada.
    // Función: El nombre de la función o método que fue llamado.
    // Argumentos: Los argumentos que fueron pasados a la función o método

    // Registrar los errores en el archivo de log, creamos un string con el mensaje que enviaremos al fichero
    $detallesError = "Error: " . $e->getMessage() . "\n" .
        "Código de error: " . $e->getCode() . "\n" .
        "Archivo: " . $e->getFile() . "\n" .
        "Línea: " . $e->getLine() . "\n";
    // funcion error_log() escribe un mensaje de error en el log del servidor, 3 indica que el mensaje de error se escribirá en un archivo
    error_log($detallesError, 3, "miFicheroErrores.log");
    exit;
} finally {
    // $resultado es un objeto PDOStatement que contiene los resultados de la consulta. Cursor es el nombre que recibe.
    // $conexion es un objeto PDO que representa la conexión a la base de datos
    // Cerrar el cursor, liberar recursos y evitar bloqueos
    $resultado->closeCursor();
    // Cerrar la conexión
    $conexion = null;
}

```

Clase PDO

Los dos métodos siguientes nos permiten ejecutar instrucciones sql, devolviendo ambos un objeto de la clase `PDOStatement`.

```
public query ( string $statement ) : PDOStatement
```

```
public prepare ( string $statement [,array $driver_options = array()] ) : PDOStatement
```

Una instancia de `PDOStatement` se crea cuando se llama a:

```
PDO->query()
```

```
PDO->prepare()
```

Muy importante

Solo debemos utilizar **query()** en consultas simples sin condicionales ni variables, es muy inseguro y permite inyección sql.

Ejemplo: obtener un objeto PDOStatement

```
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);

$conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$sql='select id, nombre, clave from usuarios';

$enlaceDatos = $conexion->query($sql);//$enlaceDatos es un objeto de PDOStatement
```

Clase PDOStatement

CLASE PDOStatement

Esta clase es **la que maneja las sentencias SQL y devuelve los resultados.**

► La consulta de datos se realiza mediante los métodos:

PDOStatement::fetch

- método que obtienen la siguiente fila de un conjunto de resultados

PDOStatement::fetchAll

- devuelve todos los resultados en un solo array.

Antes o durante la llamada a fetch o fetchAll hay que especificar cómo se quieren devolver los datos.

CLASE PDOStatement

Formato de los datos devueltos por la base de datos

PDO::FETCH_BOTH: el valor por defecto. Devuelve un array indexado cuyos índices son tanto el **nombre de las columnas** como **números**, la primera columna de la tabla se numera con 0.

PDO::FETCH_ASSOC: devuelve un array indexado cuyos índices son el **nombre de las columnas**.

PDO::FETCH_NUM: devuelve un array indexado cuyos índices son **números que se corresponden con las columnas**.

PDO::FETCH_OBJ: devuelve un objeto anónimo con nombres de atributos que corresponden con los nombres de las columnas.

```
<?php

const HOST='localhost';
const DATABASE='probando';
const USERNAME='root';
const PASSWORD='';
const PORT='3306';//puerto en el que escucha MariaDB
const CHARSET='UTF8';

try{
// $dsn = 'mysql:host=localhost;dbname=pruebas;port=3306;charset=utf8';
    $conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql=SELECT id, nombre, email FROM usuarios';
    $enlaceDatos = $conexion->query($sql);

    while ($fila = $enlaceDatos->fetch(PDO::FETCH_ASSOC)) {
        echo $fila['id'] . "<br>";
        echo $fila['nombre'] . "<br>";
        echo $fila['email'] . "<br>";
    }
}catch (PDOException $e){
    echo " <pre><br>Error: " . $e->getMessage();
    echo $cadena = print_r($e->getTrace(), true). "</pre>";
    exit();
} finally {
    // Cerrar el cursor, liberar recursos y evitar bloqueos
    $enlaceDatos ->closeCursor();
    // Cerrar la conexión
    $conexion = null;
}

?>
```

Constante PDO::FETCH_CLASS y el método setFetchMode(PDO::FETCH_CLASS...)

Es la piedra angular de la manipulación de objetos en PDO.

Crea una instancia de una clase con un nombre dado, asignando las columnas devueltas a las propiedades(atributos) de la clase.

Este modo se puede utilizar para obtener una sola fila o un array de filas de la base de datos, con fetch o fetchAll() pero es recomendable utilizar el método `setFetchMode(PDO::FETCH_CLASS...)`



CLASE PDOStatement, formato de los datos devueltos por la base de datos.

PDO::FETCH_CLASS es una constante de PDO que se utiliza para configurar el modo de obtención de los resultados de una consulta.

Cuando se utiliza PDO::FETCH_CLASS, PDO crea una nueva instancia de la clase especificada para cada fila de resultados y asigna los valores de las columnas a las propiedades correspondientes de la clase.

PDO::FETCH_PROPS_LATE es una constante en PHP que se utiliza junto con **PDO::FETCH_CLASS** para controlar cómo se crean las instancias de la clase cuando se obtienen los resultados de una consulta.

```
$enlaceDatos->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, 'Usuario', [null, null, null]);
```

Cuando se utiliza **PDO::FETCH_CLASS** solo, PDO crea:

1. Una nueva instancia de la clase especificada por cada fila de la consulta.
2. Luego asigna los valores de las columnas a las propiedades de la clase.
3. Finalmente llama al constructor de la clase.

Si se utiliza **PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE**, el orden cambia:

1. PDO crea una nueva instancia de la clase por cada fila de la consulta.
2. Después llama al constructor de la clase.
3. Finalmente asigna los valores de las columnas a las propiedades de la clase.

Diferencia entre `PDO::FETCH_CLASS` y `PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE`

- `PDO::FETCH_CLASS`: en este caso PDO asigna los valores de las columnas a las propiedades de la clase **antes** de llamar al constructor de la clase. Esto significa que, si la lógica en el constructor depende de los valores de las propiedades, esa lógica puede no funcionar como se espera **porque las propiedades aún no se han establecido cuando se llama al constructor.**
- `PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE`: en este caso PDO llama al constructor de la clase antes de asignar los valores de las columnas a las propiedades de la clase. **Esto asegura que cualquier lógica en un constructor que dependa de los valores de las propiedades funcionará como se espera.**
- Conclusión:
 - Si existe un constructor de la clase que necesita trabajar con las propiedades que se establecen a partir de los resultados de la consulta, se debe usar `PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE`.
 - Si el constructor no depende de las propiedades, puedes usar `PDO::FETCH_CLASS` solo.

```
$enlace->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, 'Usuarios',  
['parametro1', 'parametro2', 'parametro3' ]);  
//Parámetros son los valores pasados al constructor  
$enlace->execute();  
  
//Recorriendo el conjunto de datos devueltos por la consulta uno a uno con fetch  
//Los datos siguen en un buffer de la BD no han sido volcados a la memoria del ordenador  
while ($objeto = $enlace->fetch()) {  
    var_dump($objeto);  
}
```

PDOStatement::setFetchMode

```
public PDOStatement::setFetchMode(int $PDO::FETCH_CLASS, string $classname, ?array $ctorargs= NULL): bool
```

Establece el modo de obtención para esta sentencia, el modo de obtención debe ser una de las constantes PDO::FETCH_*

classname

nombre de la clase

ctorargs

argumentos del constructor de esa clase, deben ir todos en un array

PDO::FETCH_CLASS

Independientemente del método que elija, todas las columnas devueltas se asignarán a las propiedades de la clase correspondiente de acuerdo con las siguientes reglas:

1. Si hay un atributo de la clase, cuyo nombre es el mismo que el de una columna, el valor de la columna se asignará a este atributo.
2. Si no existe tal atributo, se llamará al método mágico `__set()` y se ejecutarán las instrucciones allí indicadas.
3. Si el método `__set()` no está definido para la clase, entonces se creará una propiedad pública y se le asignará un valor de columna.
4. Si deseamos que una propiedad no existente en la clase no sea asignada al objeto, basta con incluir el método `__set()` en la clase, pero vacío: `public function __set($name, $value) {}`
5. **PDO puede asignar valores a las propiedades privadas**, sorprendente, pero muy útil.
6. PDO asigna el valor a los atributos de la clase antes de llamar a un constructor. Para modificar este comportamiento, hay que utilizar `PDO::FETCH_PROPS_LATE`
7. `PDO::FETCH_PROPS_LATE` es un modificador de `PDO::FETCH_CLASS`, es decir, que debe ir junto a `PDO::FETCH_CLASS`, no puede ir solo, puesto que no se indicaría el constructor que se debe ejecutar.

Método PDOStatement::fetchObject

```
public PDOStatement::fetchObject(?string $class = "stdClass", array $constructorArgs = []): object|false
```

Obtiene la siguiente fila de una consulta y la devuelve como un objeto

class El nombre de la clase creada y si no se le pasa este argumento, devolverá un objeto anónimo de la clase stdClass

constructorArgs Los elementos de este array son pasados al constructor de la clase

Problemas:

- ▶ No permite llamar al constructor de clase antes de asignar las propiedades de la clase.
- ▶ No se puede obtener un array con todas las filas devueltas por la consulta. Trabaja fila a fila.

Ventajas:

- ▶ Si no tengo una clase creada, utiliza stdclass y obtengo un objeto anónimo
- ▶ Si la clase creada no tiene constructor que asigne valores a las propiedades de la clase, es una buena opción.

```
while ($usuario = $enlaceDatos->fetchObject('Usuario')) {  
    echo "<pre>";  
    print_r($usuario);  
    echo "</pre>";  
    // Mostrar los resultados con sintaxis de objetos  
    echo "Id: {$usuario->id} - Nombre: {$usuario->nombre} - Clave: {$usuario->clave}<br>";  
}
```

//La clase usuario no tiene constructor

//Si tuviese constructor no debe inicializar los atributos o no funcionará como esperas.

```
class Usuario  
{  
    public $id;  
    public $nombre;  
    public $clave;  
}
```

`bindParam()`

`bindValue()`

Consultas preparadas

- ▶ Muchas de las bases de datos admiten el concepto de sentencias preparadas o parametrizadas.
- ▶ Una sentencia preparada es una plantilla de SQL que las aplicaciones quieren ejecutar, y pueden ser personalizadas utilizando parámetros variables.

- ▶ *select nombre from prueba where clave='campo1'*
- ▶ *select nombre from prueba where clave=?*
- ▶ *select nombre from prueba where clave=:variable*

- ▶ Las sentencias preparadas ofrecen dos grandes beneficios:
 1. La consulta sólo necesita ser analizada (o preparada) una vez, pero puede ser ejecutada muchas veces con diferentes parámetros.
 2. Los parámetros para las sentencias preparadas **no necesitan estar entrecomillados**; el controlador automáticamente se encarga de esto. Si una aplicación usa exclusivamente sentencias preparadas, el desarrollador puede estar seguro de que **no hay cabida para inyecciones de SQL** (sin embargo, si otras partes de la consulta se construyen con datos de entrada sin escapar, aún es posible que ocurran ataques de inyecciones de SQL).

Las funciones `bindParam` y `bindValue` en PDO se utilizan para asociar valores a los parámetros en una sentencia SQL preparada.

Aunque ambas funciones parecen similares, tienen diferencias importantes en su comportamiento y uso.

► `bindParam()`

- **Asociación por Referencia:** `bindParam()` asocia el parámetro a una variable por referencia. Esto significa que el valor de la variable se evalúa en el momento de la ejecución de la sentencia, no en el momento de la llamada a `bindParam`.
- **Uso Típico:** Se utiliza cuando necesitas asociar una variable cuyo valor puede cambiar antes de la ejecución de la sentencia.

► `bindValue()`

- **Asociación por Valor:** `bindValue()` asocia el parámetro a un valor específico en el momento de la llamada. El valor no puede cambiar después de la llamada a `bindValue`.
- **Uso Típico:** Se utiliza cuando tienes un valor fijo que no cambiará antes de la ejecución de la sentencia.

```
// Ejemplo con bindParam
```

```
$sql = "SELECT * FROM usuarios WHERE id = :id";  
$stmt = $conexion->prepare($sql);  
$id = 1;  
$stmt->bindParam(':id', $id, PDO::PARAM_INT);  
$id = 2; // Cambiamos el valor de $id antes de ejecutar  
$stmt->execute();  
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);  
var_dump($usuarios);
```

```
// Ejemplo con bindValue
```

```
$sql = "SELECT * FROM usuarios WHERE id = :id";  
$stmt = $conexion->prepare($sql);  
$id = 1;  
$stmt->bindValue(':id', $id, PDO::PARAM_INT);  
$id = 2; // Cambiamos el valor de $id, pero no afecta a la sentencia preparada  
$stmt->execute();  
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);  
var_dump($usuarios);
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    //Consulta preparada. Instrucción SQL INSERT para la tabla usuarios
    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);

    // Valores de los parámetros
    $nombre = 'Pepito';
    $clave = '963';

    // Asignar valores a los parámetros y especificar los tipos de datos
    $stmtInsert->bindParam(':nombre', $nombre, PDO::PARAM_STR);
    $stmtInsert->bindParam(':clave', $clave, PDO::PARAM_STR);

    // Ejecutar la consulta
    $stmtInsert->execute();
} catch (PDOException $e) {
    echo "<br>Error: " . $e->getMessage();
    echo "<br>Línea del error: " . $e->getLine();
    echo "<br>Archivo del error: " . $e->getFile();
}
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    // Instrucción SQL INSERT para usuarios enviando un array al método execute()

    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);
    //NO puedo especificar los tipos de datos
    //No es necesario incluir bindParam y es una consulta preparada
    $datos = ['nombre' => 'Pepito2', 'clave' => '852'];
    $stmtInsert->execute($datos);
} catch (PDOException $e) {
    echo "<br>Error: " . $e->getMessage();
    echo "<br>Línea del error: " . $e->getLine();
    echo "<br>Archivo del error: " . $e->getFile();
}
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);

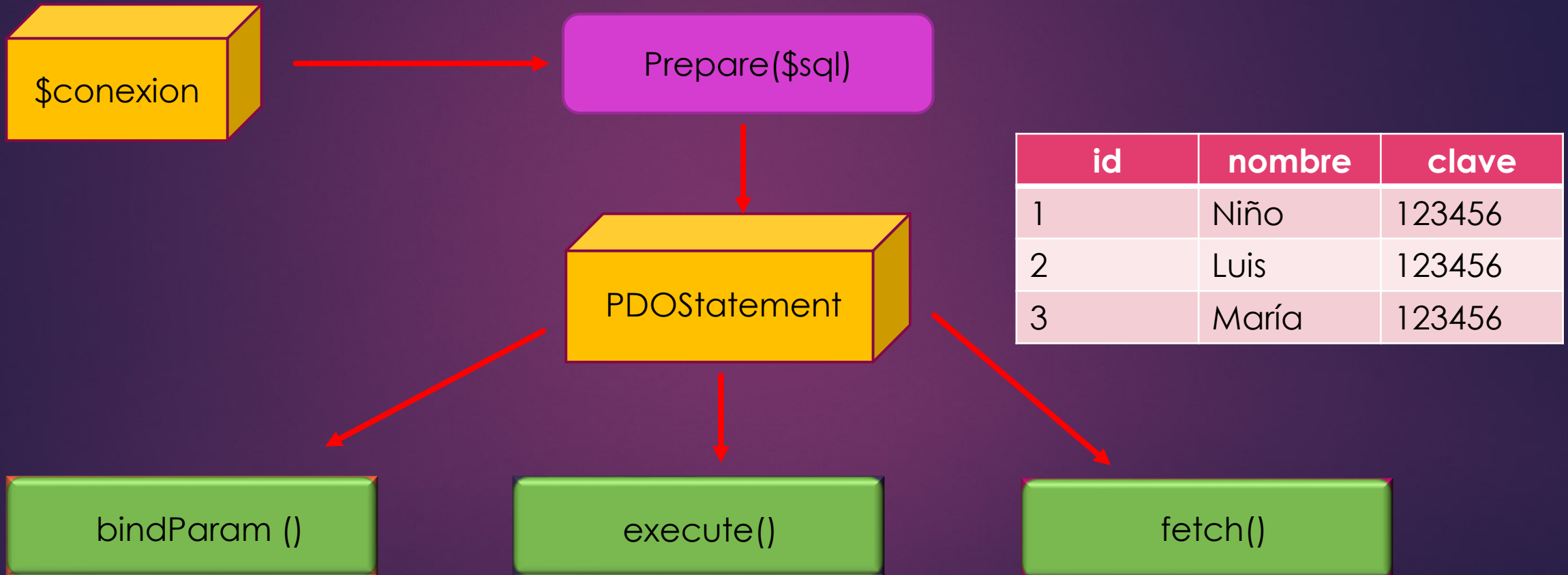
    $usuarios = [
        ['nombre' => 'Usuario1', 'clave' => 'Clave1'],
        ['nombre' => 'Usuario2', 'clave' => 'Clave2'],
        ['nombre' => 'Usuario3', 'clave' => 'Clave3'],
        ['nombre' => 'Usuario4', 'clave' => 'Clave4'],
        ['nombre' => 'Usuario5', 'clave' => 'Clave5']
    ];

    foreach ($usuarios as $usuario) {
        $stmtInsert->bindParam(':nombre', $usuario['nombre'], PDO::PARAM_STR);
        $stmtInsert->bindParam(':clave', $usuario['clave'], PDO::PARAM_STR);
        $stmtInsert->execute();
    }

    //Otra opción: No puedo indicar los tipos de datos, pero no es necesario incluir bindParam()
    foreach ($usuarios as $usuario) {
        $stmtInsert->execute($usuario);
    }
} catch (PDOException $e) {
    echo "<br>Error:{$e->getMessage()}<br>Línea del error: {$e->getLine()} <br>Archivo del error: {$e->getFile()} " ;
}
```

```
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
```

```
$sql="select id, nombre, clave from usuarios where clave=? and id>? ";
```



Consultas preparadas o parametrizadas con PDO

- La clase PDOStatement es la que trata las sentencias SQL.
- Una instancia de PDOStatement **se crea** cuando se llama a `PDO->prepare()`
- Con ese objeto creado se llama a métodos como `bindParam()` para pasar valores o `execute()` para ejecutar sentencias.
- PDO facilita el uso de sentencias preparadas en PHP, que mejoran el rendimiento y la seguridad de la aplicación.
- Cuando se obtienen, seleccionan, insertan, borran o actualizan datos, **el esquema es:**

PREPARE -> [BIND] -> EXECUTE-> [FETCH]

Consultas preparadas o parametrizadas con PDO

- Se pueden indicar los parámetros en la sentencia sql preparada de dos formas:

1. Mediante un interrogante ?

```
$sql="select id, nombre, clave from usuarios where clave=? and id=? ";
```

2. Con un nombre específico :nombre

```
$sql="INSERT INTO clientes (nombre, ciudad) VALUES (:nombre, :ciudad)";
```


Consultas preparadas o parametrizadas con PDO

Utilizando interrogantes para los valores

```
//Prepare
$enlace_datos = $conexion->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (?, ?)");
// Bind
$nombre = "Peter";
$ciudad = "Madrid";
$enlace_datos->bindParam(1, $nombre);
$enlace_datos->bindParam(2, $ciudad);
// Execute
$enlace_datos->execute();
```

Consultas preparadas o parametrizadas con PDO

Utilizando un nombre específico para las variables

```
// Prepare
$enlace_datos = $conexion->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (:nombre, :ciudad)");
// Bind
$nombre = "Charles";
$ciudad = "Valladolid";
$enlace_datos->bindParam(':nombre', $nombre);
$enlace_datos->bindParam(':ciudad', $ciudad);
// Execute
$enlace_datos->execute();
```

Método bindParam()

► El método `bindParam()` aceptan un **tercer parámetro**, que define el tipo de dato que se espera y también la longitud máxima de ese dato.

► Los ***tipos de datos*** más utilizados son:

- `PDO::PARAM_BOOL` (*booleano*)
- `PDO::PARAM_NULL` (*null*)
- `PDO::PARAM_INT` (*integer*)
- `PDO::PARAM_STR` (*string*)

```
$enlace_datos->bindParam(':edad', $edad, PDO::PARAM_INT);
```

```
$enlace_datos->bindParam(':nombre', $nombre, PDO::PARAM_STR, 12);
```



```
SELECT * FROM tabla LIMIT [OFF_SET,] N;
```

- ▶ La palabra clave **limit** se usa para limitar el número de filas devueltas en un resultado de consulta, se puede usar con select, update, delete.
- ▶ **N** es cualquier número entero que comienza desde 0, poniendo 0 el límite no devuelve ningún registro en la consulta.
- ▶ N=5 devolverá cinco registros. Si los registros en la tabla especificada son menores que N, entonces todos los registros de la tabla consultada se devuelven en el conjunto de resultados y no se producirán errores.
- ▶ El valor **OFF_SET**, número entero, nos permite especificar en qué fila(registro) comenzar la recuperación de datos. El número de registro inicial es 0 (no 1)

```
SELECT * FROM tabla LIMIT [OFF_SET,] N;
```

- La cláusula LIMIT en MySQL se utiliza para restringir el número de filas que se devuelven en una consulta. Permite especificar cuántas filas se deben devolver a partir del comienzo del conjunto de resultados.

```
SELECT columnas  
FROM tabla  
LIMIT número_de_filas;
```

número_de_filas: El número máximo de filas que deseas que se devuelvan.

- La cláusula LIMIT junto con OFFSET se utiliza para la paginación de resultados en consultas SQL. La combinación de estas dos cláusulas permite seleccionar un rango específico de filas.

```
SELECT *  
FROM productos  
ORDER BY id_producto  
LIMIT 10 OFFSET 20;
```

LIMIT 10 indica que solo se deben devolver 10 filas.

OFFSET 20 indica que se debe comenzar desde la fila número 20 (registro número 20 que cumple la condición select)

Importante:

- El primer registro que es **0**.
- La sintaxis "**LIMIT 20, 10**" en MySQL **está obsoleta** y fue reemplazada por "**LIMIT 10 offset 20**".

- ▶ El **tercer parámetro de** `bindParam()` que define el tipo de dato, es necesario cuando se utiliza la cláusula **limit** de sql e **interrogantes** para los valores.
- ▶ Puesto que **php** envía el dato como un string y **sql** necesita un entero.
- ▶ Sin embargo, si utilizamos nombres de variables **:nombre**, no habrá ningún problema.

```
$inicio = 0;
$cantidad = 3;
$enlace = self::$conexion->prepare('SELECT * FROM usuarios LIMIT ?,?');

//Cuidado con la cláusula limit, espera como valores números enteros (int),
php los pasa como string

//Es obligatorio especificar el tipo de datos para que realice un casting,
en caso contrario, se producirá un error

$enlace->bindParam(1, $inicio, PDO::PARAM_INT);
$enlace->bindParam(2, $cantidad, PDO::PARAM_INT);
```

```
$inicio = 0;
$cantidad = 3;
$enlace = self::$conexion->prepare('SELECT * FROM usuarios LIMIT :inicio, :cantidad);

//La cláusula limit, espera como valores números enteros (int), php los pasa como string
//Con esta sintaxis no hay ningún problema
$enlace->bindParam(':inicio', $inicio);
$enlace->bindParam(':cantidad', $cantidad);
```


PDO con la cláusula SQL LIKE

La cláusula SQL LIKE.

► Podemos pensar que una consulta de este tipo es válida:

```
$enlace = self::$conexion->prepare('SELECT nombre FROM usuarios where nombre like %?%');
```

- Esto `%?%` producirá un error.
- Para comprender el problema, hay que entender que un marcador `?` sólo puede representar un dato literal completo, es decir, una cadena o un número.
- **Y de ninguna manera puede representar una parte de un literal o alguna parte arbitraria de SQL.**
- Por lo tanto, cuando se trabaja con LIKE, **tenemos que preparar nuestro literal completo primero, y luego enviarlo a la consulta.**



```
self::$conexion = Conectar::conexion();  
$buscar = '%d%'; //Muy IMPORTANTE  
//Cuidado con la cláusula like, necesita ser construida antes de enviarla a prepare  
$enlace = self::$conexion->prepare('SELECT nombre FROM usuarios where nombre like ?');  
$enlace->bindParam(1, $buscar);
```

- ▶ Es el método `execute()` es el que realmente **envía o recoge los datos a la base de datos**.
- ▶ Si no se llama a `execute()` no se obtendrán los resultados sino un error.

Método execute()

```
public PDOStatement::execute ( [ array $input_parameters ] ) : bool
```

- Ejecuta la **sentencia preparada** y devuelve true o false según el resultado de la ejecución
- Si esta sentencia incluía parámetros, se debe:
 1. O llamar a **PDOStatement::bindParam()** para vincular variables o valores (respectivamente) a los marcadores de parámetros y posteriormente al método execute().
 2. O bien **pasar un array con los valores de los parámetros** al método execute() sin necesidad de llamar a bindParam().

Método execute()

```
<?php
/* Ejecutar una sentencia preparada proporcionando un array de valores de
inserción */
$calorias =150;
$color='red';
$enlace_datos =$conexion->prepare('SELECT name, colour, calories
FROM fruit WHERE calories = ? AND colour = ? ');
/*No hay método bindParam()*/
$enlace_datos->execute(array($calorias,$color));
```

Métodos PDO muy útiles

PDO::exec()

PDO::exec(). Ejecuta una sentencia SQL y devuelve el número de filas afectadas. Devuelve el número de filas insertadas, borradas o actualizadas.

No devuelve resultados de una sentencia SELECT:

```
// La siguiente instrucción devuelve 1 si que se ha eliminado un registro

echo $filas_afectadas = $conexion->exec("DELETE FROM Clientes WHERE
nombre='Luis'");

// No devuelve el número de filas si la instrucción sql es un SELECT
// Ejemplo: Devuelve 0 si ejecutó, aunque existan registros

echo $filas_afectadas = $conexion->exec("SELECT * FROM Clientes");
```

Métodos PDO muy útiles

PDO::lastInsertId ()

Este método devuelve el id autoincrementado del último registro en esa conexión
Se debe ejecutar sobre un objeto de la clase PDO, no sobre un objeto de la clase PDOStatement

public **PDO::lastInsertId ()**

```
$enlace_datos = $conexion ->
prepare("INSERT INTO usuarios (nombre) VALUES (:nombre)");
$nombre = "Angelina";
$enlace_datos->bindValue(':nombre', $nombre);
$enlace_datos->execute();
echo $id = $conexion->lastInsertId();
```

Métodos PDOStatement

public PDOStatement::rowCount (void) : int

Devuelve el número de filas afectadas por la última sentencia SQL

public PDOStatement::rowCount (void) : int

```
$enlace_datos= $conexion->prepare("SELECT * FROM usuarios");  
$enlace_datos->execute();  
echo $numero_filas=$enlace_datos->rowCount();
```


Métodos PDOStatement

public PDOStatement::fetchColumn ([int \$column_number = 0]) : mixed

Devuelve una única columna de un conjunto de resultados.
La columna se indica con un entero, empezando desde cero.
Si no se proporciona valor, obtiene la primera columna.

```
$enlace_datos = $conexion->prepare("SELECT * FROM Usuarios");  
$enlace_datos->execute();  
while ($columna= $enlace_datos->fetchColumn(1)) {  
    echo "Nombre: $columna <br>";  
}
```

Cerrar la conexión y liberar variables

Libera la variable que contiene los datos recuperados de la BD

/ La siguiente llamada a closeCursor() podría ser necesaria para algunos controladores */*

```
$enlace_datos->closeCursor();
```

MUY IMPORTANTE: cierra la conexión con la BD:

```
$conexion = NULL;
```

Cerrar la conexión y liberar variables

Cerrar el cursor libera los recursos asociados con el objeto PDOStatement y permite que la sentencia SQL sea ejecutada de nuevo.

1. **Reutilización de la Sentencia:** Si planeas ejecutar la misma sentencia SQL varias veces, cerrar el cursor permite que la sentencia sea ejecutada de nuevo sin problemas.
2. **Liberación de Recursos:** Cerrar el cursor libera los recursos del servidor de base de datos asociados con la consulta, lo cual es importante para la eficiencia y el rendimiento.
3. **Evitar Bloqueos:** En algunos sistemas de bases de datos, no cerrar el cursor puede llevar a bloqueos o a un uso excesivo de recursos.

Transacciones en BD

- Las transacciones son un conjunto de operaciones que se tratan como una sola unidad de trabajo.
- Si todas las operaciones se completan con éxito, los cambios se confirman y se hacen permanentes en la base de datos.
- Si alguna de las operaciones falla, todos los cambios se deshacen, y la base de datos vuelve a su estado anterior a la transacción.
- Las transacciones son importantes en las aplicaciones que requieren consistencia en la base de datos, como los sistemas bancarios, donde una operación de transferencia de dinero implica una operación de débito y una operación de crédito.
- En PHP, puedes trabajar con transacciones en MySQL utilizando los siguientes métodos del **objeto PDO**:
 - ✓ `beginTransaction()`: Inicia una transacción.
 - ✓ `commit()`: Confirma una transacción, haciendo permanentes todos los cambios en la base de datos.
 - ✓ `rollback()`: Revierte una transacción, deshaciendo todos los cambios en la base de datos.
- Debes usar `setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION)` para configurar PDO para que lance excepciones cuando ocurra un error. Esto es útil para manejar errores en las transacciones.
- En el código, inicias una transacción `beginTransaction()`, luego realizas una operación de débito y una operación de crédito, por ejemplo. Si ambas operaciones se completan con éxito, confirmas la transacción con `commit()`. Si alguna de las operaciones falla, reviertes la transacción con `rollback()`.

Transacciones en BD

Ejemplo: Creamos una base de datos Banco con una sola tabla llamada Cuentas:

```
CREATE DATABASE banco;
```

```
USE banco;
```

```
CREATE TABLE cuentas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    saldo DECIMAL(10, 2) NOT NULL  
);
```

```
INSERT INTO cuentas (nombre, saldo) VALUES  
('Carlos', 1000.00),  
('Ana', 2000.00),  
('Luis', 3000.00),  
('María', 4000.00);
```

```

<?php
//Base datos Banco
//Tabla cuentas
$usuario = 'root';
$contrasena = '';
$moneto = 1000;
$idOrigen = 1;
$idDestino = 2;

```

```

try {
$dsn = 'mysql:host=localhost;dbname=banco;port=3306;charset=utf8mb4';

$conexion = new PDO($dsn, $usuario, $contrasena);
$conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
// Iniciar la transacción
$conexion->beginTransaction();
// Realizar la operación de débito
$sqlDebito = 'UPDATE cuentas SET saldo = saldo - :moneto WHERE id = :id_origen';
$stmtDebito = $conexion->prepare($sqlDebito);
$stmtDebito->execute(['moneto' => $moneto, 'id_origen' => $idOrigen]);
//Verificar si la operación de débito afectó alguna fila
if ($stmtDebito->rowCount() == 0) {
    throw new PDOException('La cuenta de origen no existe');
}
// Realizar la operación de crédito
$sqlCredito = 'UPDATE cuentas SET saldo = saldo + :moneto WHERE id = :id_destino';
$stmtCredito = $conexion->prepare($sqlCredito);
$stmtCredito->execute(['moneto' => $moneto, 'id_destino' => $idDestino]);
if ($stmtCredito->rowCount() == 0) {
    throw new PDOException('La cuenta de destino no existe');
}
// Confirmar la transacción
$conexion->commit();
echo "Transacción realizada con éxito";
} catch (PDOException $e) {
    // Si algo sale mal, revertir la transacción
    $conexion->rollBack();
    echo "Error en la transacción";
    echo "<br>Error:{$e->getMessage()}<br>Línea del error: {$e->getLine()} <br>Archivo del error: {$e->getFile()} ";
}

```

id	nombre	saldo
1	Carlos	1000.00
2	Ana	2000.00
3	Luis	3000.00
4	María	4000.00